DOCUMENTATION · PROJECT

Technology stack

The latest version is at anantharajuc.github.io/QueryAPIGate-website/Guide.dc.html?d=TECHNOLOGY_STACK. This copy was made on 4 November 2026.

Technology Stack

Every entry below was cross-checked against `pyproject.toml`'s real constraints and the actual installed version in this environment (`pip show <package>`) - not written from memory. "Installed" is what this development environment resolved; "Constraint" is what `pyproject.toml` actually requires.

Language and framework

TECHNOLOGY	VERSION (THIS ENVIRONMENT)	DESCRIPTION
Python	3.12.3 (project supports 3.11-3.14, `requires-python = "≥3.11"`)	The language the whole backend is written in.
Flask	3.1.1 (constraint: `≥2.2`)	The HTTP server and routing layer - every endpoint in `app.py` is a Flask view.
Werkzeug	3.1.3 (Flask's own dependency, not pinned directly by this project)	The WSGI toolkit Flask is built on; `ProxyFix` (trusting `X-Forwarded-*` behind a reverse proxy) is used directly from it in `app.py`.

Core runtime dependencies (always installed)

LIBRARY	VERSION (CONSTRAINT)	USED FOR
PyYAML	6.0.2 (`≥6.0`)	The `format=yaml` output converter (`formats.py`).
openpyxl	3.1.5 (`≥3.1`)	The `format=xlsx` output converter (`formats.py`).

Per-database driver extras (each its own optional install)

EXTRA	LIBRARY	VERSION (CONSTRAINT)	USED FOR
`mysql`	mysql-connector-python	9.3.0 (`⩾8.0`)	The MySQL driver behind the `mysql` connection type.
`postgres`	psycopg2-binary	2.9.10 (`⩾2.9`)	The PostgreSQL driver behind the `postgres` connection type.
`clickhouse`	clickhouse-driver	0.2.9 (`⩾0.2`)	The ClickHouse driver behind the `clickhouse` connection type.
(none - stdlib)	`sqlite3`	Python's own standard library	The SQLite driver behind the `sqlite` connection type - also what the generated example database (`examples.py`) and this project's own test suite run against, since it needs no server.
`h2`	JayDeBeApi + JPype1	1.2.3 / 1.5.2 (`⩾1.2` / `⩾1.4`)	The Python ↔ JDBC bridge behind both the `h2` connection type and a generic `jdbc` connection (bring-your-own vendor `.jar`) - needs a real Java runtime on the host, not just the Python package.
(bundled with `h2`)	H2 database engine (`.jar`)	2.2.224 (`queryapigate/lib/h2-2.2.224.jar` , shipped in the wheel via `[tool.setuptools.package-data]` ; not committed - fetched from Maven Central or a mirror of it against a pinned SHA-256 by `.github/scripts/fetch_h2_jar.py` , which the Dockerfile runs too)	The actual JDBC driver JAR for the `h2` connection type - only this one vendor's driver is bundled; a generic `jdbc` connection supplies its own.
`duckdb`	duckdb	1.5.5 (`⩾1.0`)	The DuckDB driver - an embedded analytical database that also reads Parquet/CSV/JSON files on disk, S3, GCS, R2 or the web (`duckfiles.py` , locked to each connection's `allowed_paths`), and writes `format=parquet` output for every database type (`parquet.py`); the `⩾1.0` floor is specifically because `Connection.interrupt()` / `InterruptException` (relied on for query timeouts) was verified from that version on.
`mongo`	pymongo	4.18.2 (`⩾4.0`)	The MongoDB driver behind the `mongo` connection type - find-only in this version (BACKLOG #36), no aggregation/write support yet.

Feature-specific extras

EXTRA	LIBRARY	VERSION (CONSTRAINT)	USED FOR
`flow`	sqlglot	25.34.1 (`⩾23, <26`)	Real, multi-dialect SQL parsing (`sqlflow.py`) - the Access tab's table/join extraction and node-link diagram, the Access map's table filter, and real `sqlglot` -formatted SQL pretty-printing (BACKLOG #40). Deliberately never consulted by the write-guard (`sqltools.validate_sql`) - a parse failure here only affects a visualization, never query execution.
`encryption`	cryptography	50.0.1 (`⩾41`)	Encrypting a connection's password at rest (Fernet, `store.py` 's `_get_fernet()`) when `QUERYAPIGATE_SECRET_KEY` is set - optional; a connection's password is stored as given otherwise (with a startup warning).
`redis`	redis (redis-py)	8.1.0 (`⩾5`)	When `QUERYAPIGATE_REDIS_URL` is set: the response cache (`cache_ttl` , `rediscache.py`) and the rate limits (`ratelimit.py` , an atomic token bucket), shared by every instance and process. A cache failure is a miss and a rate-limit failure falls back to per-process counting - never a failed request.
`jwt`	PyJWT[crypto]	`⩾2.14`	Verifying signed-in users' tokens (`jwtauth.py`) when `QUERYAPIGATE_JWT_SECRET` or `QUERYAPIGATE_JWT_JWKS_URL` is set.
`mcp`	mcp	`⩾1.10, <2`	The MCP server (`queryapigate mcp` , `mcp_server.py`) - saved queries as tools for AI agents.

Optional extras for running/documenting the server

EXTRA	LIBRARY	VERSION (CONSTRAINT)	USED FOR
`server`	gunicorn	*(not installed in this dev environment; constraint `⩾21`)*	The recommended production WSGI server - `queryapigate serve` itself uses Flask's built-in dev server, fine for local/low-traffic use but not recommended for production, where gunicorn (or another real WSGI server) fronts the app instead.
`docs`	mkdocs-material	9.7.7 (`⩾9,<10`)	Builds the documentation site (`mkdocs.yml`) published to GitHub Pages - every page under `documentation/` (this one included) is pulled in as a snippet, not duplicated.

Dev/test tooling (not shipped to users)

TOOL	VERSION	USED FOR
`unittest` (stdlib)	Python's own standard library	The actual test runner - `python -m unittest discover -s tests -t .`, about 1,300 tests across `tests/`. Not `pytest` - a deliberate choice to add no test-framework dependency at all.
ruff	0.16.8 (`≥0.5`)	Linting (`ruff check .`) - see `[tool.ruff]` in `pyproject.toml` for the exact rule set.
mypy	2.3.1 (`≥1.10`)	Static type checking (`mypy queryapigate`) over a gradually-typed codebase, at `python_version = "3.11"` - the oldest version the project supports, so code needing a newer one fails here.
hypothesis	6.168.0 (`≥6.100`)	Property-based fuzz testing of the SQL guard (`tests/test_sql_guard_fuzz.py`) - generating adversarial SQL text to try to sneak a second statement or a write past `sqltools.validate_sql`.
openapi-spec-validator	0.9.0 (`≥0.7`)	Validating that `/openapi.json`'s generated document is actually a well-formed OpenAPI spec, not just JSON that looks plausible.
coverage	7.16.1 (`≥7.4`)	Test coverage measurement (`[tool.coverage.run]`).
Playwright	`@playwright/test` (`frontend/package.json`)	End-to-end tests of the built Console in Chromium against a real server (`frontend/e2e/`), run in CI. A frontend dev dependency only - never part of the Python package.

The frontend: the Console

The admin UI (`/console`) is the QueryAPIGate Console, in `frontend/`: React 19 and TypeScript, built with Vite, with TanStack Query for server state, React Router, CodeMirror 6 for the SQL editor, and a client generated from the backend's own OpenAPI description (`openapi-typescript` and `openapi-fetch`). No component library: the stylesheet is the hand-written one the earlier admin page had (`src/styles/classic.css`). Tests: Vitest with Testing Library, and Playwright end to end. See ADR 0001.

The build is committed to nothing: release builds (the wheel and the Docker image) run `npm run build` and ship the result in `queryapigate/console_dist/`, so `pip install queryapigate` still needs no Node.js. A source checkout without a build serves a short "not built" page at `/console`.

It replaced a single hand-written HTML/CSS/JavaScript page in a Python string (`queryapigate/ui.py`), which was removed once every screen had moved over; `/ui` redirects to `/console`.

Packaging and CI (not a runtime dependency, but part of the project)

TECHNOLOGY	USED FOR
Docker	`Dockerfile` (`debian:trixie-slim` base, with python-build-standalone's Python 3.13, installed by a pinned `uv`, and the packages in `/opt/venv` with no `pip` left in the image, ADR 0031) - an optional `WITH_H2=true` build arg adds a JRE and the bundled H2 driver for the image.
GitHub Actions	Five workflows (`.github/workflows/`): `ci.yml` (tests and lint across supported Python versions, PostgreSQL with Redis, the rolling upgrade from the previous release, the Console end to end, the Compose demo), `frontend.yml` (the Console's type check, tests and build), `release.yml` (PyPI publish, gated by `.github/scripts/check_release.py`'s tag/CHANGELOG checks), `docker.yml` (image publish), `docs.yml` (deploys the MkDocs site to GitHub Pages).
GitHub Pages	Hosts the built documentation site (see the `docs` extra above).

How it fits together

The `queryapigate` package is a Flask application, split into focused modules rather than one large file:

MODULE	RESPONSIBILITY
`app.py`	Flask app factory, request hooks, and the runtime routes (`/q/<name>` , `/execute_sql` , `/catalog` , `/events` , …).
`v1.py` , `v1_spec.py`	The Management API (`/api/v1`) routes, and their OpenAPI description.
`services/`	What each Management API operation means - queries, connections, access (keys and roles), administrators, collections, audit, MCP - kept apart from HTTP.
`governance.py`	What every front door (REST, MCP, the events server) applies: authentication, rate limits, metrics.
`admins.py` , `adminroles.py`	Named administrators and their admin tokens; the four admin roles and the capability each `/api/v1` operation needs (ADR 0003).
`jwtauth.py`	Signed-in users' tokens: verification, roles from claims, `from_claim` parameters.
`engine.py`	Looks up the connection, validates the SQL, runs it, wraps the page of results.
`runners.py`	One driver class per database type: connect, bind parameters in that driver's own style, run, reset.
`pool.py`	A thread-safe pool of idle connections (reset on return, health-checked, discarded after errors).
`sqltools.py`	SQL validation (single statement, read-only unless permitted - including writes hidden behind a read-only keyword), pagination, `:name` bound-parameter and `{name}` text-substitution handling.
`sqlflow.py`	Best-effort real SQL parsing (via `sqlglot`) for the Access tab's table/join diagram and pretty-printing - never consulted by the write-guard.
`mongotools.py`	The JSON-document equivalent of `sqltools.py` for Mongo saved queries - placeholder substitution and the `$where` -rejection safety check.
`schema.py`	Schema introspection (tables/columns/PK-FK markers) and a table's real DDL, one query per dialect.
`formats.py`	The `ResultSetDTO` and every output converter (JSON/NDJSON/CSV/TSV/XML/YAML/XLSX/Parquet).
`parquet.py`	Parquet output, written by DuckDB with column types taken from the values.
`duckfiles.py`	DuckDB file access: each connection locked to its `allowed_paths` , object-storage credentials, views and automatic views over files.
`tableguard.py`	`allowed_tables` enforcement, with a real SQL parser (`sqlglot`).
`store.py`	Connections, saved queries and their versions, the audit log - kept in the metadata store; optional at-rest password encryption.
`db.py`	The metadata store itself: SQLite (`queryapigate.db`) or a shared PostgreSQL database, the schema and its upgrades, `migrate-to-postgres` .
`history.py`	Run history: batched writes, retention, `GET /api/v1/history` .

MODULE	RESPONSIBILITY
`events.py`, `broadcast.py`	Live events (Server-Sent Events) - on the main server and the standalone `queryapigate events` server.
`alerts.py`	The live conditions behind the Alerts screen and `GET /api/v1/alerts`.
`instances.py`	Every process using the store, for the instance list and its alerts.
`mcp_server.py`	The MCP server (`queryapigate mcp`): saved queries as tools for AI agents.
`apikeys.py`	API keys and roles: creation, permission checks (`can_use_query` / `can_use_collection` /...), the single source of truth for what a key can reach.
`definitions.py`	Validating a saved query's own definition (SQL or Mongo shape, declared parameters) - the same validation path for saving via the UI, the API, and loading the bundled examples.
`params.py`	Resolving and type-checking a saved query's declared parameters against the values a caller actually sent.
`collection_admin.py`	Moving/renaming a saved query's collection and reporting the access-grant impact of doing so.
`bundle.py`	Import/export of a collection as a portable JSON bundle.
`postman.py`	Exporting a collection as a Postman collection file.
`cache.py`	The response cache backing a saved query's `cache_ttl`.
`ratelimit.py`	Per-key rate limiting - hand-rolled, no third-party rate-limiting library.
`cors.py`	CORS header handling - hand-rolled, no `flask-cors` dependency.
`metrics.py`	The `/metrics` Prometheus-text endpoint and the counters/gauges/histograms it renders.
`logging_setup.py`	Structured request logging.
`examples.py`	The bundled example APIs (BACKLOG #27) - a generated SQLite database, four scenarios plus a cross-collection role, install/remove/status.
`openapi.py`	The generated OpenAPI document and the Swagger UI page.
`config.py`	All environment-variable configuration in one place.
`databases.py`, `experimental.py`, `deprecations.py`	The support matrix, what is experimental, and what is deprecated - each a single list the docs and tests check against.
`errors.py`	The single `ApiError` exception every endpoint raises for a client-facing failure.
`cli.py`	The `queryapigate` command (`serve`, `mcp`, `events`, `init`, `backup`, `migrate-to-postgres`, `export`, `collection ...`, `examples ...`, `admins ...`).
`console.py`	Serves the prebuilt Console (`console_dist/`) at `/console`, with its Content-Security-Policy.